

Autonomous Systems

Decision Making for Mobile Robot Navigation using AlphaBot2

Group 18

Presentation 1

David Gaspar - 106541

Francisco Valério - 106533

Miguel Carneiro - 106218

Pedro Mónico - 106626

- Introduction and Motivation
 - What is Decision Making in Robotics
 - Real World Applications
- Markov Decision Processes
 - Formalization
 - Solving the MDP
- Reinforcement Learning
 - Definition
- Hardware
 - Alfabot 2
 - Project Integration
- Workplan
- Robot Setup and Early Testing

Introduction and Motivation

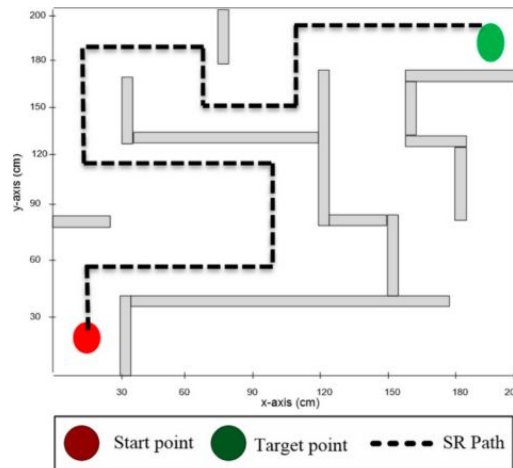
Process of selecting the best action for a robot to achieve a goal. Operates in sequential decision problems (not single-step decisions) and must handle uncertainty in perception and motion

Objective → Maximize cumulative long-term reward = Minimize cumulative long-term cost

Core Challenges

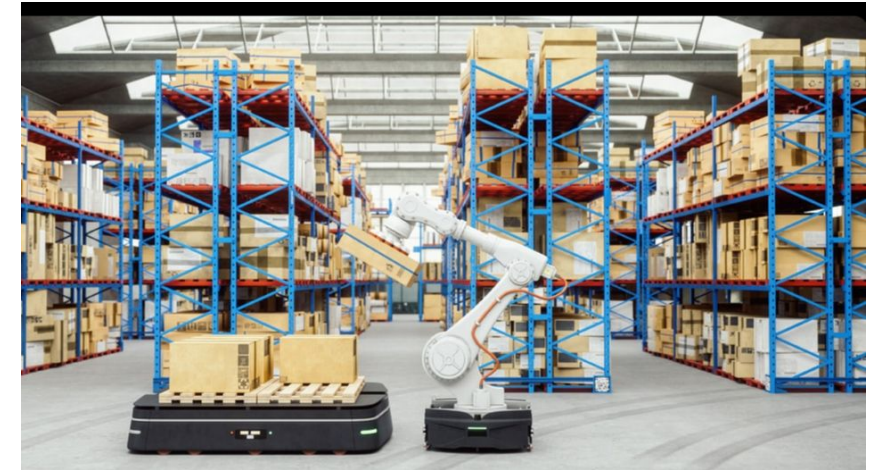
- Robot motion is uncertain (motor noise, slippage)
- Sensor information is imperfect (limited field of view, noise)
- If no real odometry, position must be estimated indirectly

Navigation in Mazes



- Finding a path from A to B in an unknown or partially known environment.
- The state space is relatively small (grid positions), the reward structure is simple.

Warehouse Automation



- Continuously executes tasks, and adapts to a changing environment.
- The state space is larger. Includes location, cargo status, shelf inventory, and time constraints.

Markov Decision Processes

A **Markov Decision Process (MDP)** is a formal mathematical model for decision making under uncertainty, where outcomes are partly random and partly controlled by the agent.

It is defined by:

- S - Set of possible **states** the robot can be in;
- A - Set of **actions** available to the robot;
- $T(s, a, s')$ - **Transition probability** of reaching s' from s after action a ;
- $R(s, a)$ - **Reward** received for taking action a in state s ;
- $\gamma \in [0,1]$ - **Discount factor**, balancing immediate vs future rewards.

The future depends only on the current state, not the history:

$$p(s_{t+1} \mid s_t, a_t)$$

A **policy** π maps states to actions telling the robot what to do in every situation.

To find the optimal policy π^* , we first compute the value function $V^*(s)$ (a measure of how good it is to be in state s assuming optimal behavior from that point onward) which is defined recursively by the **Bellman Equation**:

$$V^*(s) = \max_a \left[R(s, a) + \gamma \sum_{s'} T(s, a, s') V^*(s') \right]$$

There are 2 classical approaches to solve it:

- **Value Iteration**: repeatedly update the value of every state until convergence;
- **Policy Iteration**: start with any policy, evaluate it, improve it and repeat until optimal.

Reinforcement Learning

An agent learns to make decisions by interacting with an environment since it **has not prior knowledge** of it.

Unlike in MDPs, the policy must be **learned from experience** rather than computed from a **known model**.

Reinforcement Learning →

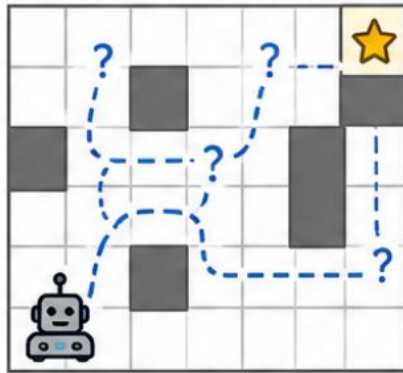
When **transition model** $T(s, a, s')$ is **unknown**

Robot learns optimal behavior through **trial and error**

No prior knowledge of environment dynamics required

EXPLORATION

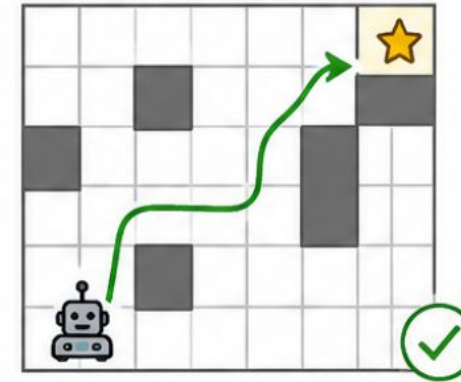
(Try new actions)



Try new actions to discover better strategies

EXPLOITATION

(Use learned knowledge)



Use current best-known actions to maximize reward

Hardware



AlphaBot2

Main Components

Raspberry Pi 3 B

Omni-direction wheel

ST188: Reflective infrared photoelectric sensor

2 chargeable batteries (5V port)

5M pixels Camera

2x N20 micro gear motor

The environment is modeled as a grid where each cell represents a possible robot state (position, velocity, acceleration, and orientation). The goal cell provides positive rewards, while obstacles carry negative penalties.

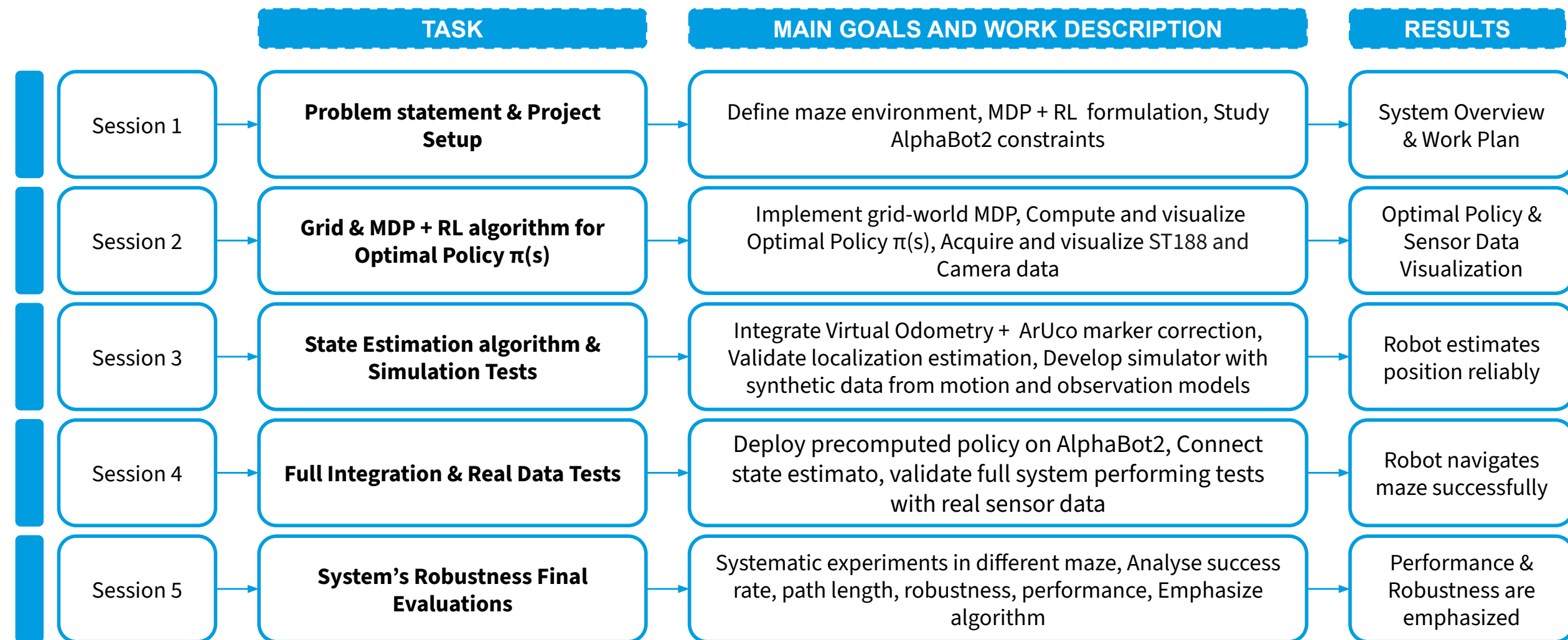
State Estimation Pipeline →

Virtual odometry tracks motion between steps but **accumulates drift over time**

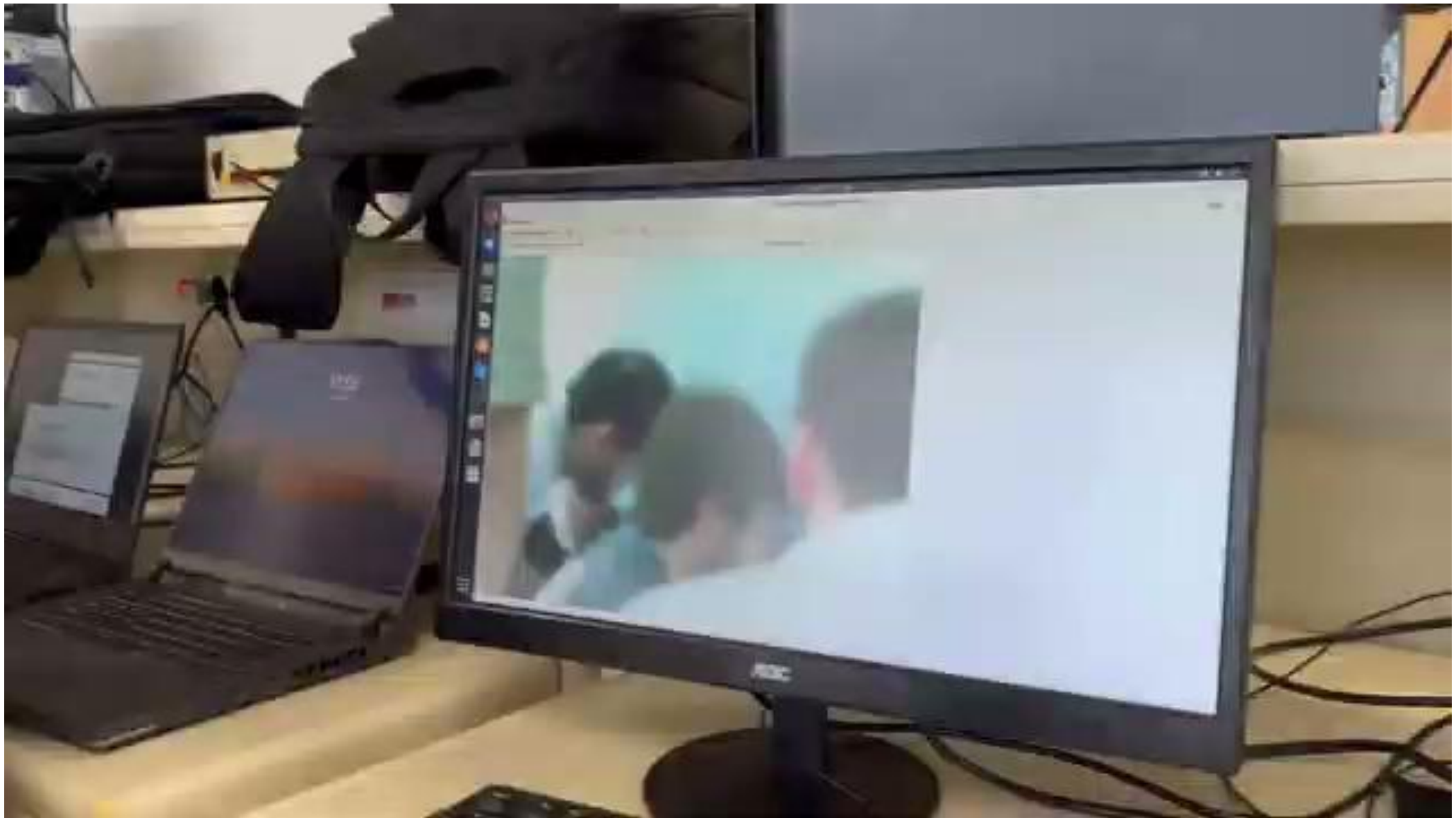
Vision System provides **position corrections** with the information from the **Visual Markers**

Sensor Fusion combines both sources to maintain accurate state estimates **for policy execution**

Work Plan



Robot Setup and Early Testing





Thank You!

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

$Q(s, a)$	α	r	γ	$\max_{a'} Q(s', a')$	$Q(s', a')$
Current value	Learning rate ($0 < \alpha \leq 1$)	Reward received	Discount factor ($0 \leq \gamma < 1$)	Best future value	Value of next state-action

Q-Learning Algorithm →

Learns action-value function $Q(s,a)$ directly from experience

Updates Q-values based on observed rewards and transitions

Converges to optimal policy Q^* given sufficient exploration

